

```
# FREE OFFLINE RAG-BASED PDF READER

import streamlit as st
from sentence_transformers import SentenceTransformer
import faiss
from PyPDF2 import PdfReader
import numpy as np
from gpt4all import GPT4All

# APP TITLE

st.title("Offline Free RAG PDF Reader 📄🔍")
st.write("""
Upload a PDF and ask questions.
This app uses **local embeddings + FAISS retrieval + local GPT4All LLM**.
""")

# UPLOAD PDF
uploaded_file = st.file_uploader("Upload a PDF", type=["pdf"])

if uploaded_file is not None:

    # READ PDF

    pdf = PdfReader(uploaded_file)
    full_text = ""

    for page in pdf.pages:
        if page.extract_text():
            full_text += page.extract_text() + " "

    # Simple chunking
    chunks = [c.strip() for c in full_text.split(".") if len(c.strip()) > 30]

    st.success(f"PDF loaded successfully! {len(chunks)} text chunks created.")

    # LOAD EMBEDDING MODEL

    @st.cache_resource
    def load_embedding_model():
        return SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

    embed_model = load_embedding_model()

    # CREATE EMBEDDINGS

    embeddings = embed_model.encode(chunks, convert_to_numpy=True).astype("float32")

    # FAISS INDEX

    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)

    # LOAD LOCAL GPT4ALL MODEL (IMPORTANT PART)

    @st.cache_resource
    def load_llm():
        model_dir = r"F:\BS Classes\NTU\Machine learning\streamlit\model"
        model_name = "q4_0-orca-mini-3b.gguf"

        return GPT4All(
            model_name=model_name,
            model_path=model_dir,
            allow_download=False
        )

    llm = load_llm()

    # USER QUESTION

    query = st.text_input("Ask a question from the PDF:")

    if query:

        # Embed query
        query_embedding = embed_model.encode(
            [query], convert_to_numpy=True
        ).astype("float32")

        # Retrieve top 3 relevant chunks
        _, indices = index.search(query_embedding, k=3)
        retrieved_chunks = [chunks[i] for i in indices[0]]

        st.subheader("📄 Retrieved Context")
        for i, chunk in enumerate(retrieved_chunks):
            st.write(f"{i+1}. {chunk}")

        # GENERATE ANSWER (RAG)

        prompt = f"""
Answer the question using ONLY the context below.

Context:
{chr(10).join(retrieved_chunks)}

Question:
{query}

Answer:
"""

        with st.spinner("Generating answer..."):
            answer = llm.generate(prompt, max_tokens=300)

        st.subheader(" Generated Answer")
        st.write(answer)
```